

Village by CA Website

Technical Documentation

Table of Contents

- User Stories and Mockups
 - User Stories ----- 3
 - Mockups ----- 11
- System Architecture: High-level diagram -----22
- Entity Relationship Diagram -----23
- Database Schema -----24
- Sequence Diagrams
 - Creation of a Start-Up -----25
 - Creation of a User ----- 26
 - Update a Partner ----- 27
- Village by CA's Internal API -----28
- Village by CA's External API -----40
- SCM and QA Plans -----42
- Technical Justifications -----43
- Design Justifications ----- 47
- Dependencies between tasks ----- 48
- Sprint Planning Overview ----- 49
- Project Closure Report ----- 70

USERS STORIES

Must Have:

Admin Stories

As an admin, I want to add, delete and modify start-ups information so that I can update the Village's information and make the start-up visible on the website.

- Tasks
 - Adding a Startup entity to the database
 - Adding a Startup repository file to manage the CRUD operations on the startup table
 - Adding a Startup class file to validate the format of the data that we will store in our database
 - Adding a Startup Services file to manage the business logic of the startups
 - Adding a Startup controller file that will call the appropriate functions as needed for the route
 - Adding a Startup route file that will include all the routes the admin need to realise these operations and will call all the needed controllers

As an admin, I want to add, delete and modify types associated with my start-ups so that I can update the start-ups information and this way visitors can sort startups according to their type on the website.

- Tasks
 - Adding a Type entity to the database
 - Adding a Type repository file to manage the CRUD operations on the startup table
 - Adding a Type class file to validate the format of the data that we will store in our database
 - Adding a Type Services file to manage the business logic of the types.
 - Adding a Type controller file that will call the appropriate functions as needed for the route.
 - Adding a Type route file that will include all the routes the admin need to realise these operations and will call all the needed controllers.
 - Adding a types space page for the admin so they can modify their informations

As an admin, I want to post, update and delete blog articles, so that I can make the website more attractive and increase the number of visits.

- Tasks
 - Adding a Post entity to the database
 - Adding a Post repository file to manage the CRUD operations on the startup table
 - Adding a Post class file to validate the format of the data that we will store in our database
 - Adding a Post Services file to manage the business logic of the posts
 - Adding a Post controller file that will call the appropriate functions as needed for the route
 - Adding a Post route file that will include all the routes the admin need to realise these operations and will call all the needed controllers
 - Adding a posts space page for the admin so they can modify their informations

As an admin, I want to add, update and delete the partners lists and information, so that I can keep the website updated with new partnerships and make them visible on the website.

- Tasks
 - Adding a Partner entity to the database
 - Adding a Partner repository file to manage the CRUD operations on the startup table
 - Adding a Partner class file to validate the format of the data that we will store in our database
 - Adding a Partner Services file to manage the business logic of the partners
 - Adding a Partner controller file that will call the appropriate functions as needed for the route
 - Adding a Partner route file that will include all the routes the admin need to realise these operations and will call all the needed controllers
 - Adding a partners space page for the admin so they can modify their informations.

For all admin routes :

- Adding a function that generate a token with the role of the user so only the admin can do these verifications.
- Adding a function that will read the token and verify the role of the user to forbid other people except the admin to access these routes.

For all Entities that need to use pictures :

- Create a Picture entity in the database
- Configure Cloudinary to host the picture so we can manipulate them.
- Create a Cloudinary function that upload pictures to Cloudinary
- Create a multerv middleware that intercept upload files

Start-up Stories

As a Start-up, I want to update my account information, so that I can keep my information updated for the visitors.

- Tasks
 - Adding a User entity to the database.
 - Adding a User repository file to manage the CRUD operations on the startup table.
 - Adding a User class file to validate the format of the data that we will store in our database.
 - Adding a User Services file to manage the business logic of the users.
 - Adding a User controller file that will call the appropriate functions as needed for the route.
 - Adding a User route file that will include all the routes the logged-in user need to realise these operations and will call all the needed controllers.
 - Adding a Startup route file that will include all the routes the logged-in user need to realise these operations and will call all the needed controllers.
 - Adding a function that will read the token and verify the role of the user to forbid other people except the admin and the logged-in user to access these routes.
 - Adding a User route file that will include all the routes the admin need to realise these operations and will call all the needed controllers.

As a Start-up I want to know what are the partners of the association, so I can learn what are the association's network.

- Task
 - Adding a partners page for the visitors so they can see the partners informations

As a Start-up I want to write a quote that describe my experience with the association so that I can inform other start-up about the quality of the association.

- Tasks
 - Adding a Quote entity to the database
 - Adding a Quote repository file to manage the CRUD operations on the startup table
 - Adding a Quote class file to validate the format of the data that we will store in our database
 - Adding a Quote Services file to manage the business logic of the quotes
 - Adding a Quote controller file that will call the appropriate functions as needed for the route
 - Adding a Quote route file that will include all the routes the logged-in user need to realise these operations and will call all the needed controllers
 - Adding a Quote route file that will include all the routes the admin need to realise these operations and will call all the needed controllers

Visitors Stories

As a visitor, I want to take a rendez-vous to know more about how the Village and the Mayenne environment can help me in my current situation, so that I know who I can contact.

- Tasks
 - Adding a Contact transporter that will send email to the director of the association with the message of the visitor
 - Adding a Contact class file that will validate the data that want to be send to the director of the association
 - Adding a Contact services file that will contain the business logic of the contact form and use the transporter to send the email
 - Adding a Contact controller file that will use the adequate function in the Contact services
 - Adding a Contact route that everybody can access to contact the association
 - Adding a Contact form page for the visitor to use to send their email
 - Adding a field in the subject of the contact form that said « I want to know more about the Mayenne environment »

As a visitor, I want to know what are the upcoming events in the Village, so that I can take part in events that interest me.

- Task
 - Adding an Agenda page that will display all the informations about the upcoming events, and the news of the association

As a visitor, I want to discover the start-ups present in the Mayenne department so that I can contact them for a collaboration if one of them is interesting to me.

- Task
 - Adding a Startups page that display all the start-ups and their informations notably their website and email so they can discover and contact them

As a visitor, I want to know what services the Village by CA is delivering so that I can know if the Village by CA can respond to my needs.

- Task
 - Adding a field in the subject of the contact form that said « I want to know more about how the association can respond to my needs »

Should Have:

Admin stories

As an admin, I want to add, update and delete room for rent information, so that I can keep the visitors updated with available rooms

- Tasks
 - Adding a Location entity to the database
 - Adding a Location repository file to manage the CRUD operations on the startup table
 - Adding a Location class file to validate the format of the data that we will store in our database
 - Adding a Location Services file to manage the business logic of the locations
 - Adding a Location controller file that will call the appropriate functions as needed for the route
 - Adding a Location route file that will include all the routes the admin need to realise these operations and will call all the needed controllers
 - Adding a locations space page for the admin so they can modify their informations

As an admin, I want to create, delete and update new events on an agenda, so that I can keep start-ups and visitors informed of the upcoming events in the Village.

- Tasks
 - Adding a Event entity to the database
 - Adding a Event repository file to manage the CRUD operations on the startup table
 - Adding a Event class file to validate the format of the data that we will store in our database
 - Adding a Event Services file to manage the business logic of the events
 - Adding a Event controller file that will call the appropriate functions as needed for the route
 - Adding a Event route file that will include all the routes the admin need to realise these operations and will call all the needed controllers
 - Adding a events space page for the admin so they can modify their informations

Start-up Story.

As a Start-up, I want to see the upcoming events in the Village agenda, so that I can plan to come to the events that interest me.

- Task
 - Adding an Agenda page that will display all the informations about the upcoming events, and the news of the association

Visitor Story.

As a visitor, I want to rent a place so that I can organize some events here.

- Task
 - Adding a location page where I can see all the locations and their details and a field in the subject of the contact form that said « I want to rent a place »

As a visitor, I want to have a partnership with the Village by Ca so that I can help them in their activities and collaborate with them.

- Task
 - Adding a field in the subject of the contact form that said « I want to become a partner of the Village by CA »

As a visitor, I want to contact the Village headmaster, so that I can ask various questions about the events or the Village.

- Task
 - Adding a field in the subject of the contact form that said « I want to know more about the Village by CA»

Could Have:

Visitor and Start-up Story.

As a visitor I want to talk with a chatbot so that I have my answer quickly and don't bother the administrator with very simple questions.

- Task
 - Configure a basic Chatbot

Won't Have:

Start-up Story.

As a start-up, I want to have a list of the actors in the Mayenne ecosystem so that I know who can help my business.

- Task
 - Adding a page with all the actors of the Mayenne ecosystem

MOCKUPS

Index page

Start-Ups

Partenaires

Agenda

Locaux

Contact

Acompagner les entreprises de demain

Découvrez les Start-Ups →

Le Village by CA de Laval est un écosystème dynamique dédié à l'accompagnement des startups en phase de croissance. Cet espace favorise les rencontres entre jeunes entreprises, experts et partenaires pour accélérer l'innovation sur le territoire. Les startups y trouvent un soutien personnalisé, allant du mentorat à l'accès à un réseau solide d'acteurs économiques. Le Village crée ainsi un environnement propice au développement, à l'expérimentation et à la réussite entrepreneuriale.

Nous les accompagnons :

<

T.Y.M.E

LAVILLAGE

TAINT

>

b

John Doe
Boulangier

"Le Village by CA m'a offert un super accompagnement, Je n'aurais jamais pu développer mon entreprise seule !" "

CaféPro

James Philip
Co-Pro

"Le Village by CA m'a offert un super accompagnement, Je n'aurais jamais pu développer mon entreprise seule !" "

nasa

Alex Miles
NASA

"Le Village by CA m'a offert un super accompagnement, Je n'aurais jamais pu développer mon entreprise seule !" "

En savoir plus

Agenda

Juin-3	Mai-6	Mardi-8	Juin-6	Vien-7	Juin-5	Mai-4
Scherrer Elodie Cofondatrice & FARMELI Océane	KORDEINE Eliane Directrice commerciale et marketing			Renaudine Marketing	Béguez Lucile Comptable Etudiante et entrepreneuse	
	PONCHICKA Julianne Associée - Chef Bernad Rogues Chef cuisinier			Béguez Sarah Développeuse	COMTESSIE Julie Présidente & Associée	Fédération des Professionnels de la Restauration
BARTHEL Théo						

Les actualités de votre Région

Une nouvelle Start-Up intègre le Village !

Lire

Une nouvelle Start-Up intègre le Village !

Lire

Une nouvelle Start-Up intègre le Village !

Lire

Voir plus d'articles

Envie de vous faire accompagner ?

Contactez-nous directement et explorons ensemble tout le potentiel de votre entreprise.

Nous contacter

LE VILLAGE BY CA

Start-Ups
Partenaires
Agenda
Locaux
Contact

Connexion

in o

© 2024 LE VILLAGÉ BY CA - TOUS DROITS RÉSERVÉS Mentions légales

MOCKUPS

Start-ups page



Envie de vous faire accompagner ?

Contactez-nous directement et explorons ensemble tout le potentiel de votre entreprise.

Nous contacter



Start-Ups
Partenaires
Agenda
Locaux
Contact

Connexion



© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS Mentions légales

Envie de vous faire accompagner ?

Contactez-nous directement et explorons ensemble tout le potentiel de votre entreprise.

Nous contacter



Start-Ups
Partenaires
Agenda
Locaux
Contact

Connexion



© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS Mentions légales

MOCKUPS

Partners page



MOCKUPS

Events page



MOCKUPS

Posts details

[Start-Ups](#)[Partenaires](#)[Agenda](#)[Locaux](#)[Contact](#)

Actualités

[Retour →](#)

Une nouvelle Start-Up intègre le village !



Orientis vero limes in longum protentus et rectum ab Euphratis fluminis ripis ad usque supercilia porrigitur Nili, laeva Saracenis conterminans gentibus, dextra pelagi fragoribus patens, quam plagam Nicator Seleucus occupatam auxit magnum in modum, cum post Alexandri Macedonis obitum successorio iure teneret regna Persidis, efficaciae inpetrabilis rex, ut indicat cognomentum. Novitates autem si spem adferunt, ut tamquam in herbis non fallacibus fructus appareat, non sunt illae quidem repudiandae, vetustas tamen suo loco conservanda; maxima est enim vis vetustatis et consuetudinis. Quin in ipso equo, cuius modo feci mentionem, si nulla res impediatur, nemo est, quin eo, quo consuevit, libentius utatur quam intractato et novo. Nec vero in hoc quod est animal, sed in iis etiam quae sunt inanima, consuetudo valet, cum locis ipsis delectemur, montuosis etiam et silvestribus, in quibus diutius commorati sumus.

Novitates autem si spem adferunt, ut tamquam in herbis non fallacibus fructus appareat, non sunt illae quidem repudiandae, vetustas tamen suo loco conservanda; maxima est enim vis vetustatis et consuetudinis. Quin in ipso equo,

Novitates autem si spem adferunt, ut tamquam in herbis non fallacibus fructus appareat, non sunt illae quidem repudiandae, vetustas tamen suo loco conservanda; maxima est enim vis vetustatis et consuetudinis. Quin in ipso equo, cuius modo feci mentionem, si nulla res impediatur, nemo est, quin eo, quo consuevit, libentius utatur quam intractato et novo. Nec vero in hoc quod est animal, sed in iis etiam quae sunt inanima, consuetudo valet, cum locis ipsis delectemur, montuosis etiam et silvestribus, in quibus diutius commorati sumus.

Envie de vous faire accompagner ?

Contactez-nous directement et explorons ensemble tout le potentiel de votre entreprise.

[Nous contacter](#)

[Start-Ups](#)[Partenaires](#)[Agenda](#)[Locaux](#)[Contact](#)[Connexion](#)

© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS [Mentions légales](#)

MOCKUPS

Location pages

LE VILLAGE

Start-UpsPartenairesAgendaLocauxContact

Les Locaux :

Nos start-ups s'épanouissent dans nos locaux, où elles peuvent pleinement s'éprouver et se créer un réseau. C'est un cadre idéal pour rassembler et développer votre Start-up.

La Licorne :



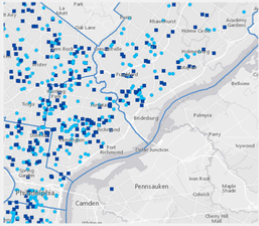
La région :

La Mayenne une région d'opportunités :

Et prima post Oedriem quam, ut dictum est, ab hac descriptione discevimus, Commagena, nunc Euphratensis, clementer adsurgit, Hierapoli, vetere Nino et Samosata civitatibus amplis iniustis.

Iam in altera philosophiae parte, quae est quaerendi ac disserendi, quae logikhi dicitur, iste vester plane, ut mihi quidem videtur, inermis ac nudus est. falsi pro vero probatum sit, sublatum esse omne iudicium veri et falsi putat.

Orientis vero limes in longum protentus et rectum ab Euphratis fluminis ripis ad usque supercilia porrigitur Nili, laeva Saracenis continens gentibus, dextra pelagi fragoribus patens, quam plagam Nicator Seleucus occupatam auxit magnum in modum, cum post Alexandri Macedonis obitum successorio iure teneret regna Persidis, efficaciae inpetrabilis rex, ut indicat cognomenitum.



Location de salles :

Le Village by CA propose des salles de réunions afin de vous retrouver pour des moments conviviaux pour renforcer la cohésion de vos équipes.



Salle à louer
Prix : 10\$/h
Espace: 30 m2
[En savoir plus](#) [Je réserve](#)



Salle à louer
Prix : 10\$/h
Espace: 30 m2
[En savoir plus](#) [Je réserve](#)



Salle à louer
Prix : 10\$/h
Espace: 30 m2
[En savoir plus](#) [Je réserve](#)

Envie d'en savoir plus sur l'écosystème mayennais ?

Contactez-nous directement et explorons ensemble tout le potentiel de votre entreprise.

[Nous contacter](#)

LE VILLAGE

Start-UpsPartenairesAgendaLocauxContact

Connexion

[in](#) [o](#)

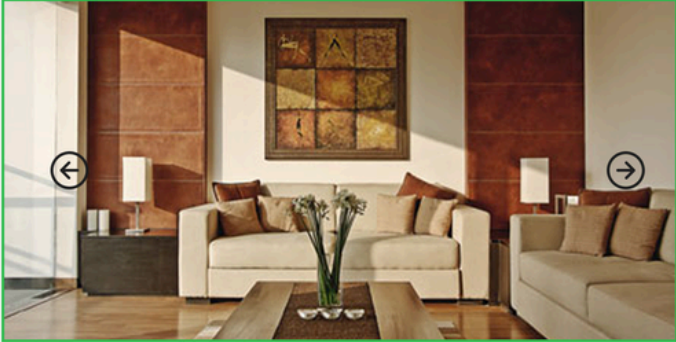
© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS Mentions légales

LE VILLAGE

Start-UpsPartenairesAgendaLocauxContact

Salle à louer

[Retour](#)



Prix : 10 \$/h

Taille : 30 m2

Orientis vero limes in longum protentus et rectum ab Euphratis fluminis ripis ad usque supercilia porrigitur Nili, laeva Saracenis continens gentibus, dextra pelagi fragoribus patens, quam plagam Nicator Seleucus occupatam auxit magnum in modum, cum post Alexandri Macedonis obitum successorio iure teneret regna Persidis, efficaciae inpetrabilis rex, ut indicat cognomenitum. Novitates autem si spem adferunt, ut tamquam in herbis non fallacibus fructus appareat, non sunt illae quidem repudiandae, vetustas tamen suo loco conservanda; maxima est enim vis vetustatis et consuetudinis. Quin in ipso equo, cuius modo feci mentionem, si nulla res impediatur, nemo est, quin eo, quo consuevit, libentius utatur quam intractato et novo. Nec vero in hoc quod est animal, sed in iis etiam quae sunt inanimata, consuetudo valet, cum locis ipsis delectemur, montuosis etiam et silvestribus, in quibus diutius commorati sumus.

Novitates autem si spem adferunt, ut tamquam in herbis non fallacibus fructus appareat, non sunt illae quidem repudiandae, vetustas tamen suo loco conservanda; maxima est enim vis vetustatis et consuetudinis. Quin in ipso equo,

[Je réserve](#)

LE VILLAGE

Start-UpsPartenairesAgendaLocauxContact

Connexion

[in](#) [o](#)

© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS Mentions légales

LE VILLAGE

16

MOCKUPS

Contact page



[Start-Ups](#) [Partenaires](#) [Agenda](#) [Locaux](#) [Contact](#)

Contact

Formulaire de contact:

Nom, Prénom :

Email :

Sujet de votre demande :

Message :

☐ I'm not a robot  [Privacy](#) - [Terms](#)

Envoyer →



[Start-Ups](#) [Partenaires](#) [Agenda](#) [Locaux](#) [Contact](#)

Connexion

© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS [Mentions légales](#)

MOCKUPS

Authentification page

[Connexion](#)

Village by CA

Espace de connexion :

Bonjour !

E-Mail :

Mot de passe :

[Se connecter →](#)



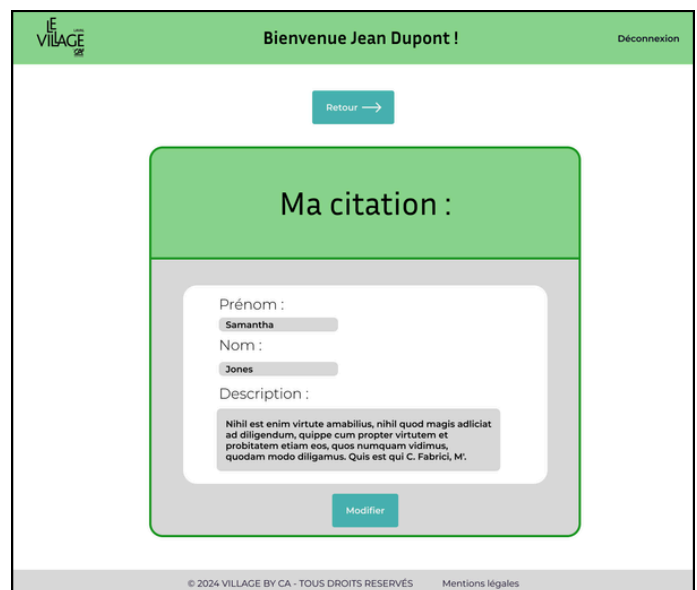
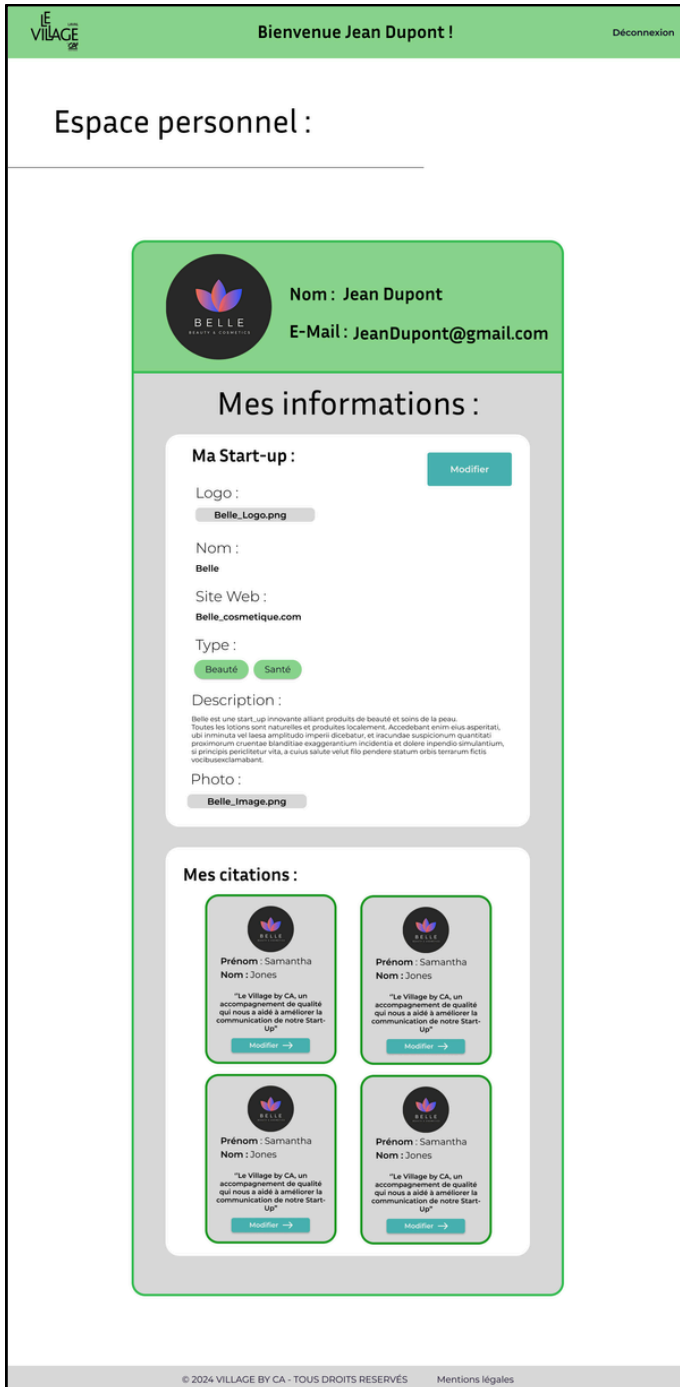
Content de vous revoir !

© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS

[Mentions légales](#)

MOCKUPS

Personal space pages



MOCKUPS

Admin page



MOCKUPS

Admin users management space



Bienvenue Amandine !

Déconnexion

Utilisateurs

Trier par :

Nom de Start-Up ▼



John Doe

Dole

Modifier



James Philip

NASA

Modifier



Alex Thompson

Green Garden

Modifier



Jean Dupond

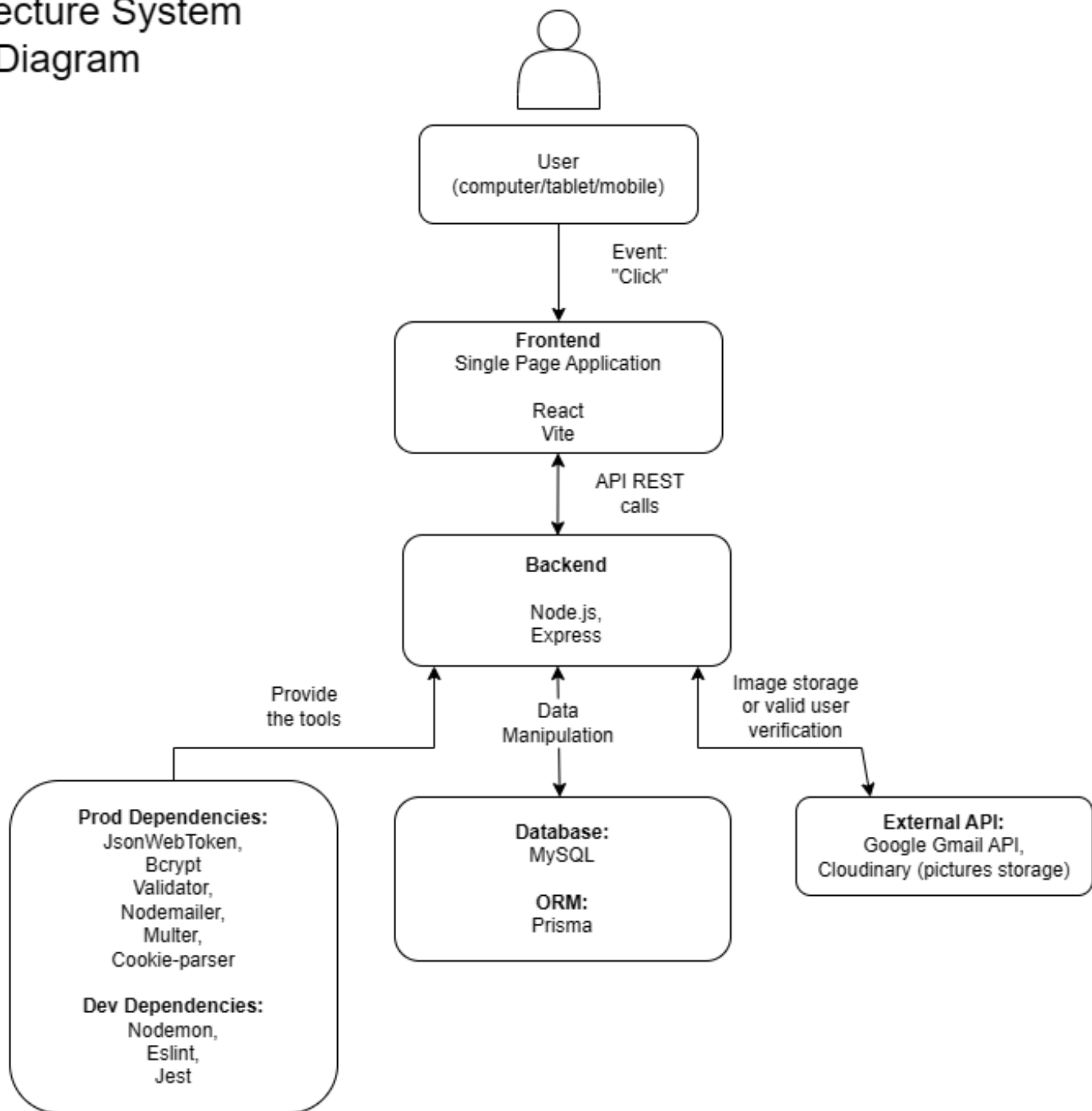
DELL

Modifier

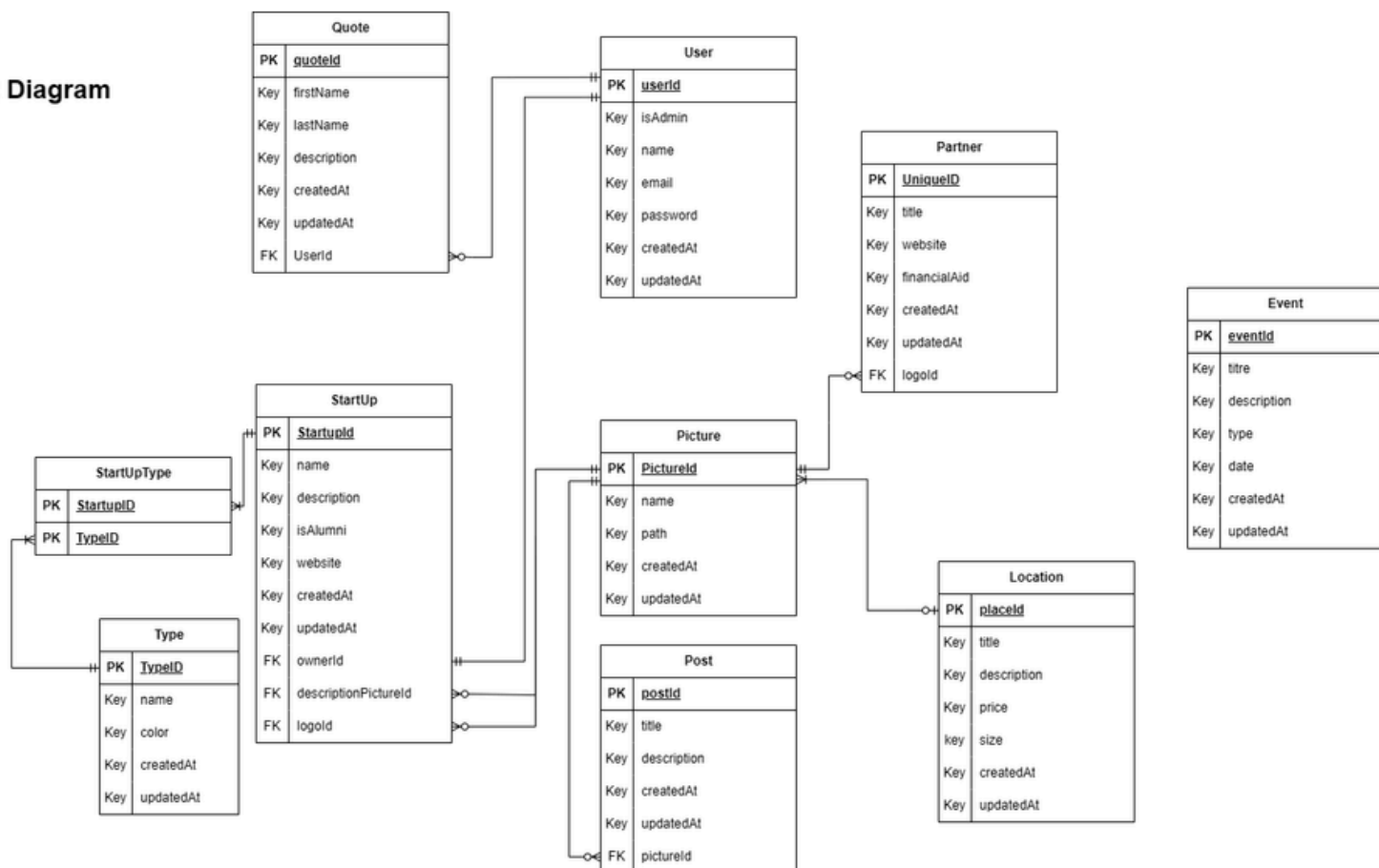
© 2024 VILLAGE BY CA - TOUS DROITS RESERVÉS

Mentions légales

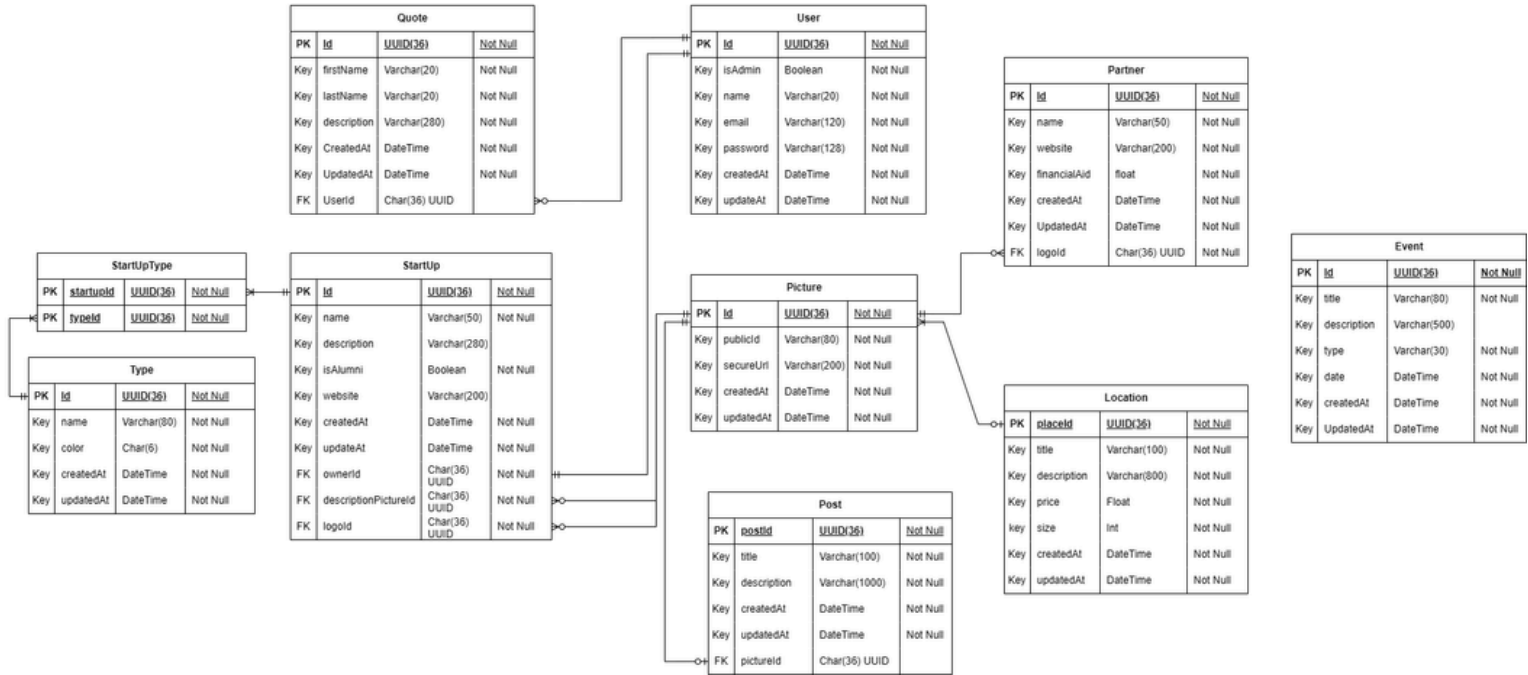
Architecture System Diagram



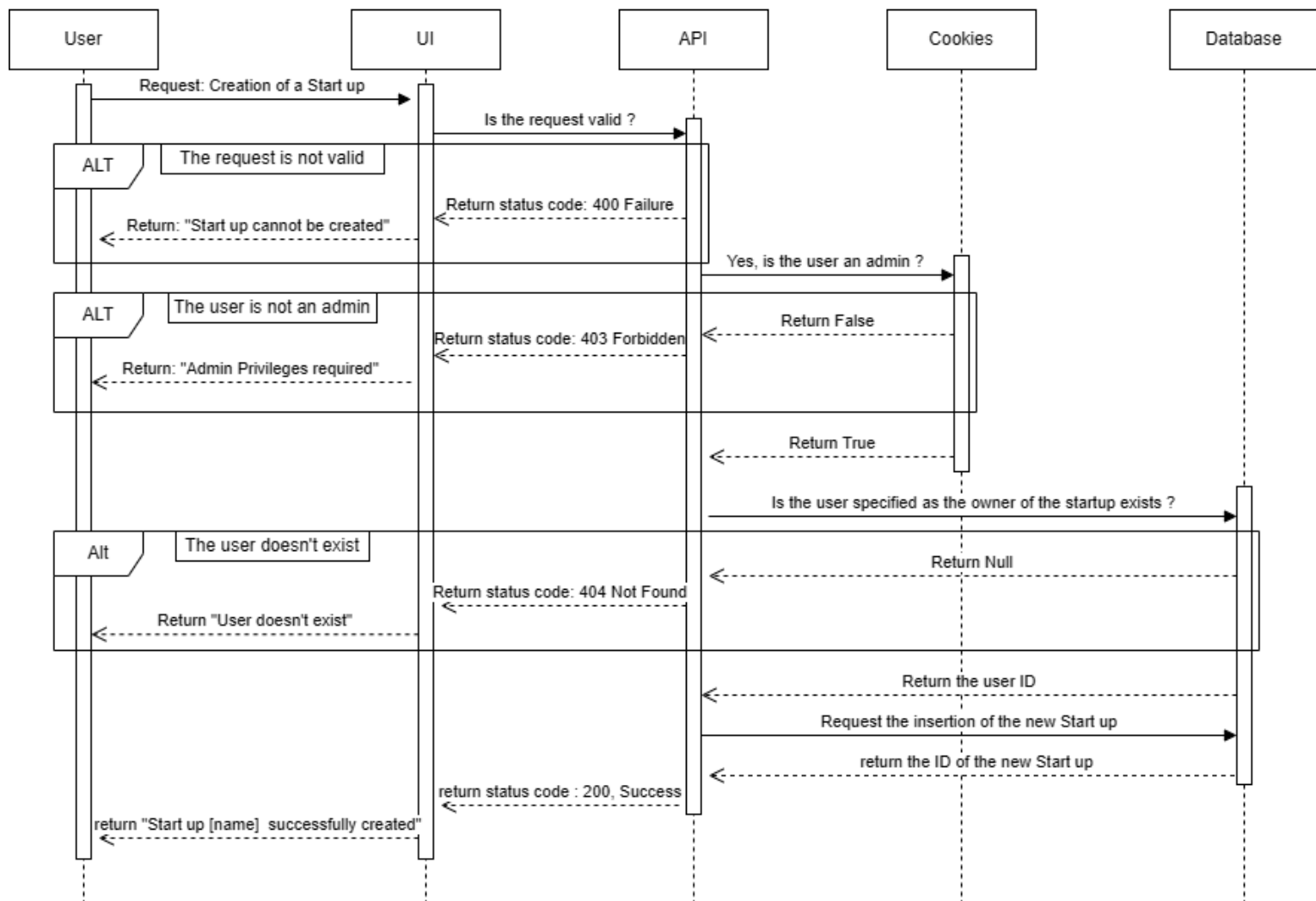
ER Diagram



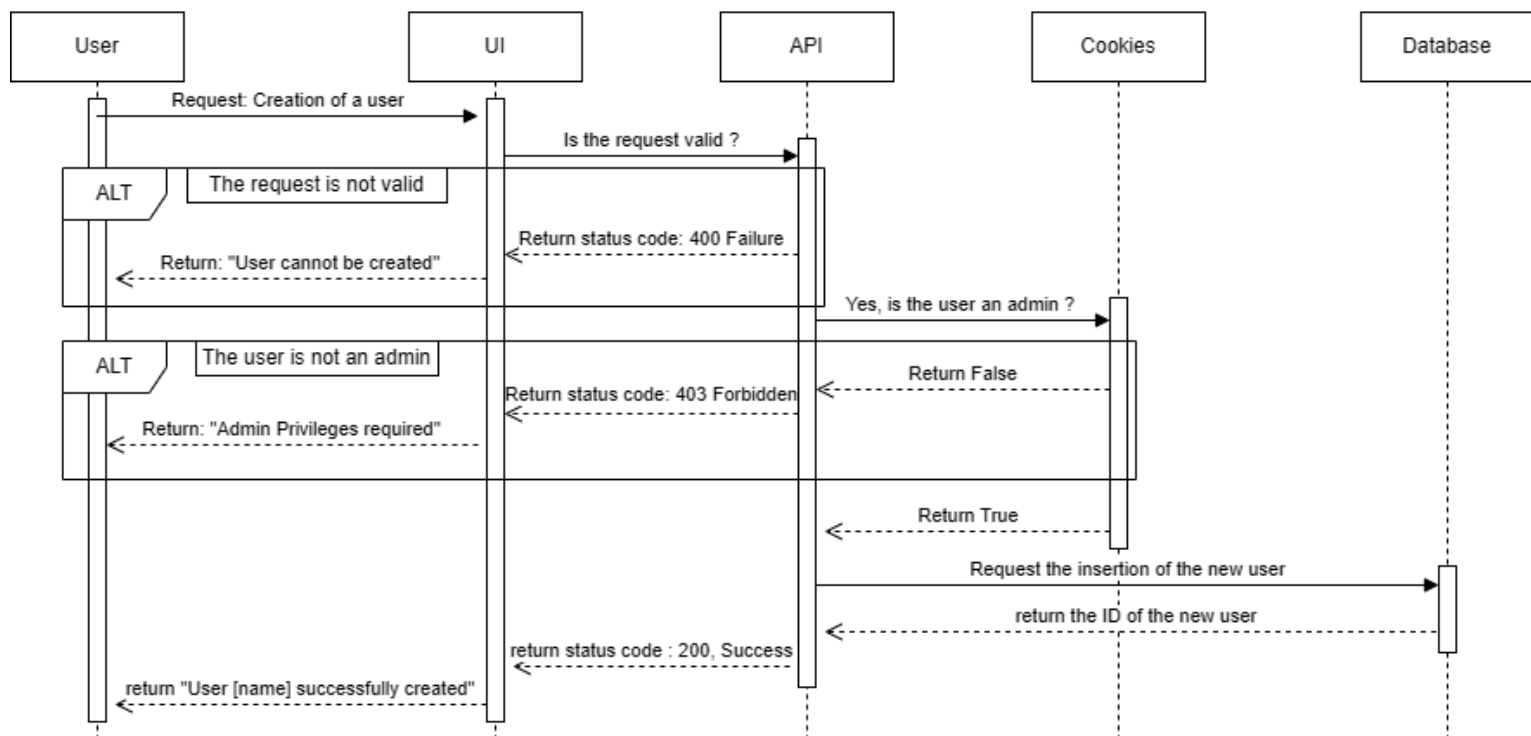
Database Schema



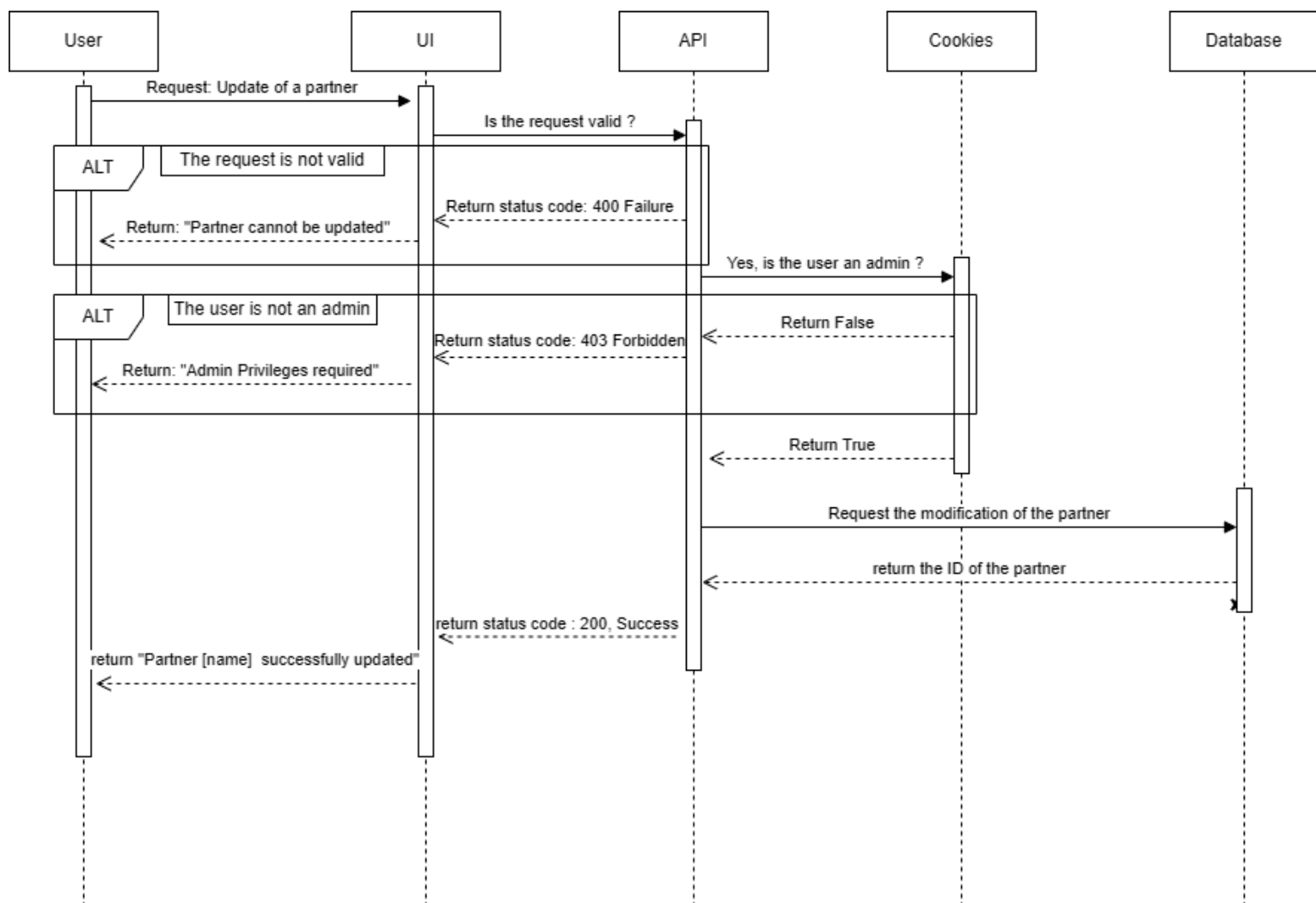
Creation of a StartUp



Creation of a User



Update a Partner



Village by CA's internal API :

For this project we use two external API, which is after the documentation of the internal API.

Below you will find all the documentation for the application's internal API.

You will find a table detailing each endpoint using the application's API:

- The route for each endpoint
- The method used for this route
- The format of the information sent to the API
- The format of the information returned by the API
- A short description of the route's purpose

Here all the API calls are used for three main purposes:

- 1.The visuals of a showcase site with data automatically updated for all visitors.
- 2.Profile pages for logged-in users.
- 3.Creation of an administrator area to manage all website informations and data easily.

There are short descriptions for each endpoint to provide details on how to access it, who will use it, how and why.

1- Endpoints for the showcase site :

Each endpoint in this category is usable by all visitors, whether logged in or not, and can only use GET methods, except for the contact form.

INDEX ENDPOINT :

The index endpoint has only one allowed method, which is a GET request, because it is only used to retrieve various pieces of information from the database in order to send them back to our front end for display.

Route	Method	Input Format	Output Format	Function
/index	GET		<pre>{ "startUps" : [{"logo" : "logo.path"}], "partners" : [{"logo" : "logo.path"}], "quotes" : [{"firstName" : "firstName", "lastName" : "lastName", "logo" : "logo.path", "description" : "description"}], "events" : [{"title" : "title", "date" : dateTime, "type" : "type", "description" : "description"}], "posts" : [{"title" : "title", "description" : "description"}]}</pre>	Display multiple pieces of information related to the association, to present what they're doing.

PARTNERS ENDPOINTS :

The partner endpoints only have one allowed method, which is a get, because they are only used to retrieve partner information with more or less detail in order to send it back to our Front for display.

/partners	GET		<pre>{ "name": "name", "logo": "<logo.path>", "financial aid": 0 }</pre>	Display all the partners
/partners/:id	GET		<pre>{ "name": "name", "website": "website", "logo": "<logo.path>", "financial aid": 0 }</pre>	Display the details of a partner

STARTUPS ENDPOINTS :

Startup endpoints only allow one method, which is a GET request, because they are only used to retrieve startup information with varying levels of detail in order to send it back to our front end for display.

Route	Method	Input Format	Output Format	Function
/startups	GET		<pre>{ "logo": "<logoPicture.path>", "name": "name", "type": ["type"] }</pre>	Display all the start-ups
/startups/:id	GET		<pre>{ "logo": "<logoPicture.path>", "name": "name", "type": ["type"], "isAlumni": False, "website": "website", "owner": "owner.id", "description": "description", "descriptionPicture": "<descriptionPicture.path>" }</pre>	Get the details of a specific start-up

LOCATIONS ENDPOINTS :

The endpoint locations only have one allowed method, which is a get, because they are only used to retrieve location information with more or less detail in order to send it back to our Front for display.

/locations	GET		<pre>{ "title": "title", "price": "price", "size": "size" }</pre>	Display all the locations
/locations/:id	GET		<pre>{ "title": "title", "description": "description", "price": "price", "size": 0, "picture": ["<picture.path>"] }</pre>	Display the details of the location

CONTACT ENDPOINT :

The only endpoint open to all visitors with a post request, the API retrieves the data sent from the Front and processes it in order to send an email to the admin with the request information.

/contact	POST	<pre>{ "email": "email", "name": "name", "subject": "subject", "text": "text" }</pre>	<pre>{ "message": "Demande de contact envoyée" }</pre>	Sending a contact request
----------	------	---	--	---------------------------

2- Endpoints for logged users :

Apart from the login route, here are all the endpoints available to all logged-in users so they can view and modify their information.

AUTH ENDPOINTS :

To access all the auth endpoints, the user has to login to their account via the route /auth/login. Once the API validates that the information are correct it creates a session token stored in a cookie on the client side, that permits the access of all the other auth routes. As you can see you must have a valid token in each route.

Route	Method	Input Format	Output Format	Function
/auth/login	POST	{ "email" : "email", "password" : "password" }	{ "message" : "Welcome back" } + Creation of a token on a cookie.	Login to the application.
/auth/profile	GET	{ Authorization : Bearer <Token> }	{ "name" : "name", "email" : "email", "quotes" : [{ "firstName": "firstName", "lastName": "lastName", "title" : "title" }], "startUp" : "name", "logo" : "<logoPicture.path>", "type" : ["type"], "website" : "website", "descriptionPicture" : "<descriptionPicture.path>", "description" : "description" }	See current user details and information.

/auth/profile/update	PATCH	<pre>{ Authorization : Bearer <Token>} {"name" : "name", "email" : "email", "startUp" : "name", "logo" : "<logoPicture.path>", "type" : ["type"], "website" : "website", "descriptionPicture" : "<descriptionPicture.pat h>", "descriptpion" : "description"}</pre>	<pre>{"message " ; "Profile updated successfully !"} </pre>	Updating the current user profile. The user can change almost all the information he sees in the GET request of auth/profile. He just cannot change his role and his password.
/auth/profile/:id	GET	<pre>{Authorization : Bearer <Token>}</pre>	<pre>{"firstName" : "fistName", "lastName" : "lastName", "title" : "title", "description" : "description"}</pre>	Get a quote details that the current have created
/auth/profile/:id	PATCH	<pre>{Authorization : Bearer <Token>} {"firstName" : "fistName", "lastName" : "lastName", "title" : "title", "description" : "description"}</pre>	<pre>{"message" : "Quote successfully updated !"} </pre>	Update a quote that the current user have created
/auth/profile/:id	DELETE	<pre>{Authorization : Bearer <Token>}</pre>	<pre>{"message" : "Quote successfully deleted"} </pre>	Delete a quote the current user have created

3- Endpoint of the admin interface :

Here are the endpoints for the administrator interface. To access them, you must be a logged-in user with the admin role and a valid session token. The API will verify the token's validity with each request and identify whether the current user is an admin.

ADMIN ENDPOINT :

Here are the user management options. The administrator can list all existing users in the database, view their detailed information, and modify it if necessary or delete a user. Only the administrator can create new users.

Route	Method	Input Format	Output Format	Function
/admin/users	GET	{ Authorization : Bearer <Token>}	{ "name" : "name", "startUp" : "<logoPicture.path>" }	Display all the users
/admin/users	POST	{ Authorization : Bearer <Token> { "name" : "name" , "email" : "email@email.com", "password" : "password" }	{ "name" : "name", "email" : "email", "message" : " User.<name> created." }	Creation of a user.
/admin/users/:id	GET	{ Authorization : Bearer <Token>}	{ "name" : "name", "email" : "email", "isAdmin" : False, "startUp" : "<logoPicture.path>" }	Get the details of a User
/admin/users/:id	PATCH	{ Authorization : Bearer <Token> { "name" : "newName" , "isAdmin " : False, "email" : "newEmail@email.com", "password" : "newPassword" }	{ "message" : "User<name> updated" }	Modification of a user.
/admin/users/:id	DELETE	{ Authorization : Bearer <Token>}	{ "message" "User successfully deleted" }	Deleting a user.

Here are the routes for managing startups. The administrator can list all existing startups in the database, view their detailed information, and modify or delete them as needed. The administrator can only assign one user as the startup owner and cannot assign a user who is already associated with another startup. Only an administrator can create a new startup.

/admin/startups	GET	{Authorization : Bearer <Token>}	{ "logo": "<logoPicture.path>", "name" : "name", "type" : ["type"]}	Display all the start-ups.
/admin/statusups	POST	{Authorization : Bearer <Token>}	{ "logo": "<logoPicture.path>", "name" : "name", "type" : ["type"], "isAlumni" : False, "website" : "website", "owner" : "owner.id"} {message : <startUp.name> created successfully !}	Creation of a new start_up Some fields like "logo", "type" or "website" are not mandatory at the creation.
/admin/statusups/:id	GET	{Authorization : Bearer <Token>}	{ "logo": "<logoPicture.path>", "name" : "name", "type" : ["type"], "isAlumni" : False, "website" : "website", "owner" : "owner.id", "description" : "description", "descriptionPicture" : "<descriptionPicture.path>"}	Get the details of a start-up.
/admin/statusups/:id	PATCH	{Authorization : Bearer <Token>}	{ "logo": "<logoPicture.path>", "name" : "name", "type" : ["type"], "isAlumni" : False, "website" : "website", "owner" : "owner.id", "description" : "description", "descriptionPicture" : "<descriptionPicture.path>"}	{ "message" : "Star-up updated successfully !"} Updating a start-up
/admin/startups/:id	DELETE	{Authorization : Bearer <Token>}	{ "message" : "Start-up deleted successfully !"} Deleting a start-up	

Here are the routes for managing partners. The administrator can list all existing partners in the database, view their detailed information, and modify or delete them as needed.

Only an administrator can create a new partner.

/admin/partners	GET	{Authorization : Bearer <Token>}	[{"name" : "name", "logo": "<logo.path>"}]	Display all the partners
/admin/partners	POST	{Authorization : Bearer <Token> {"name" : "name", "website" : "website", "logo" : "<logo.path>" "financial aid" : 0}	{"message" : "<partner.name> created successfully !"}]	Creation of a new partner
/admin/partners/: id	GET	{Authorization : Bearer <Token>}	{"name" : "name", "website" : "website", "logo" : "<logo.path>" "financial aid" : 0}	Getting the details of a partner
/admin/partners/: id	PATCH	{Authorization : Bearer <Token> {"name" : "name", "website" : "website", "logo" : "<logo.path>" "financial aid" : 0}	{"message" : "Partner successfully updated !"}]	Updating a partner
/admin/partners/: id	DELETE	{Authorization : Bearer <Token>}	{"message" : "Partner successfully deleted !"}]	Deleting a partner

Here are the routes for managing citations.

The administrator can list all existing citations in the database, view their detailed information, and delete them if necessary. All logged-in users can create a new citation.

/admin/quotes	GET	{Authorization : Bearer <Token>}	[{"firstName": "firstName", "lastName": "lastName", "title" : "title"}]	Display all the quotes
/admin/quotes/:id	GET	{Authorization : Bearer <Token>}	{"logo" : "<startUpLogo.path>", "firstName" : "fistName", "lastName" : "lastName", "title" : "title", "description" : "description"}	Display the details of a quote
/admin/quotes/:id	DELETE	{Authorization : Bearer <Token>}	{"message" : "Quote successfully deleted !"}]	Delete a quote

Here are the routes for managing events. The administrator can list all existing events in the database, view their detailed information, and modify or delete them as needed. Only an administrator can create a new event.

/admin/events	GET	{Authorization : Bearer <Token>}	[{"title" : "title", "date" : dateTime, "type" : "type"}]	Display all the events for the actual month and the next 2 months
/admin/events	POST	{Authorization : Bearer <Token>}	{ "title" : "title", "date" : dateTime, "type" : "type", "description" : "description"}	Creation of a new event
/admin/events/:id	GET	{Authorization : Bearer <Token>}	{ "title" : "title", "date" : dateTime, "type" : "type", "description" : "description"}	Display the details of an event
/admin/events/:id	PATCH	{Authorization : Bearer <Token>}	{ "title" : "title", "date" : dateTime, "type" : "type", "description" : "description"}	Updating an event
/admin/event/:id	DELETE	{Authorization : Bearer <Token>}	{"message" : "Event successfully deleted"}	Deleting an event

Here are the routes for managing posts. The administrator can list all existing posts in the database, view their detailed information, and modify or delete them as needed. Only an administrator can create a new post.

/admin/posts	GET	{Authorization : Bearer <Token>}	[{"title" : "title", "description" : "description"}]	Display all the posts
/admin/posts	POST	{Authorization : Bearer <Token> {"title" : "title", "description" : "description", "picture" : "picture.path" }	{"message" : "Post created successfully"}	Creation of a post
/admin/posts/:id	GET	{Authorization : Bearer <Token>}	{"title" : "title", "description" : "description", "picture" : "picture.path" }	Display the details of a post
/admin/posts/:id	PATCH	{Authorization : Bearer <Token> {"title" : "title", "description" : "description", "picture" : "picture.path" }	{"message" : "Post updated successfully !"}	Updating a post
/admin/posts/:id	DELETE	{Authorization : Bearer <Token>}	{"message" : "Post successfully deleted !"}	Deleting a post

Here are the routes for managing rentals. The administrator can list all existing rentals in the database, view their detailed information, and modify or delete them as needed. Only an administrator can create a new location.

/admin/location	GET	{Authorization : Bearer <Token>}	{ "title" : "title", "price" : "price", "size" : "size" }	Displaying all the locations
/admin/location	POST	{Authorization : Bearer <Token> { "title" : "title", "description" : "description", "price" : "price", "size" : 0 "picture" : ["<picture.path">] }	{ "message" : "place successfully created !" }	Creating a new location
/admin/location/:id	GET	{Authorization : Bearer <Token>}	{ "title" : "title", "description" : "description", "price" : "price", "size" : 0 "picture" : ["<picture.path">] }	Display the details of a place
/admin/location/:id	PATCH	{Authorization : Bearer <Token> { "title" : "title", "description" : "description", "price" : "price", "size" : 0, "picture" : ["<picture.path">] }	{ "message" : "place successfully created !" }	Updating an existing place
/admin/location/:id	DELETE	{Authorization : Bearer <Token>}	{ "message" : "place successfully deleted !" }	Deleting a place

Village by CA's external API :

For this project, we use only two external API: Google Gmail API and Cloudinary.

Google Gmail API:

We will use it on the /contact endpoint for communication and security purposes.

The Google Gmail API is used to securely send emails from our application to the website administrator when a visitor submits the contact form. Instead of relying on traditional SMTP credentials which can be less secure, the Gmail API uses OAuth 2.0 to authorize email sending in a controlled and protected way.

When a user submits the form, our backend collects the visitor's message and email address, then sends a request to the Gmail API to deliver the email to the admin's inbox. OAuth 2.0 ensures that only our application, with the proper access token, is allowed to perform this action. This prevents unauthorized use of the email service and ensures that sensitive operations are handled safely.

We will therefore use Nodemailer to interact with the Gmail API. Nodemailer handles the email-sending process within our application and uses OAuth 2.0 credentials to securely connect to Gmail. This allows our backend to send messages through the Gmail API without exposing any sensitive SMTP passwords, while ensuring that all email operations remain fully authenticated and protected.

Cloudinary:

We will use it on all the endpoints/routes that need to upload a picture.

Indeed, we will use Cloudinary to store our images on an external storage server, not in our database, as this would negatively impact our application's performance and quickly overload our database. We will only store the path to each image located in Cloudinary within our database.

Therefore, each time we want to save images in our application, we will make calls to the Cloudinary API to save them.

The images stored there are primarily logos of startups and partners, photos of posts, and photos of rooms available for rent from the administrator.

Each time a user uploads an image, our Multer dependency facilitates the file's upload to Cloudinary.

Multer intercepts the file, verifies that it complies with the defined rules (such as allowed size), and then forwards the file to the Cloudinary SDK.

The Cloudinary SDK automatically uses our API keys (stored in environment variables) to authenticate the request and send the image to our Cloudinary storage.

Cloudinary then stores the image in the specified folder and returns a public URL. This URL is saved in our database so we can retrieve and display the image in the application.

SCM and QA Plans

SCM Processes :

Version control : Git

Branching strategy : one branch per person, one branch for dev implementation and a production branch for the deliverable application.

Pull Request and merges : the commits of this project will follow the following template:

[changes : add, fix, remove] : [description of the new implementation].

Each merge on the development and production branch will have to be approved by the other team member through a pull request.

QA :

Code clarity : ESLint for backend

Api testing : API will be tested using Postman

Code testing: Unittest

Technical Justifications

Prod Dependencies :

REACT

First, we chose the React library for the front end, simply because it is widely used and very modern. React has a huge community with extensive documentation, and since we wanted to learn about other front-end technologies, it's the perfect tool for learning thanks to its many resources.

Furthermore, because many developers use it, and because the Village by CA association will want to update its interface in the long term, many developers will be able to easily improve it. In addition, we really like the fact that every element of the interface can be reused, as it facilitates website optimization.

NODE.JS and EXPRESS

We want to work with Node.js and Express, which allow the use of JavaScript as a backend language and faster application development with a REST API. We want to work with JavaScript because we find Python too verbose, and with Express because we also dislike the file structure imposed by Flask.

Furthermore, we have worked with Node.js and Express in class, so we are familiar with these technologies, which gives us time to learn others like React. Finally, Node.js and Express are widely used in application development and are therefore very well documented with numerous resources.

MySQL

We chose MySQL for our database because it's a very popular technology, and therefore well-documented and easy to maintain. It's an excellent tool for relational databases. MySQL is performant and more than sufficient for the anticipated data volume. Furthermore, MySQL is free and very stable, which is ideal for a non-profit organization. Combined with the Prisma ORM, which is widely used with Node.js, it makes for a powerful and easily scalable tool because if we ever need a larger database, such as PostgreSQL, Prisma makes that migration easy.

PRISMA

We specifically want to use Prisma because it offers a cleaner structure compared to other ORMs that rely on decorators, classes, and other elements scattered throughout the code. Prisma uses only a single file that describes the entire database.

Prisma allows for easy switching between database management systems (for example, moving from MySQL to PostgreSQL) without rewriting all the application logic, providing significant scalability and flexibility for the project's future.

Finally, Prisma boasts exemplary documentation, an active community, and seamless integration with Express and Node.js, making it a natural choice for a modern project seeking to combine productivity, robustness, and scalability.

Google Gmail API

We use the Gmail API to securely manage and automate email operations within the application. This integration enables the system to send messages directly to the website administrator while capturing the visitor's email address to facilitate follow-up communication. By relying on OAuth 2.0, the Gmail API ensures that all sensitive email actions are executed safely and with strong authentication guarantees.

The Gmail API offers a dependable and scalable approach to handling communication workflows, avoiding the limitations and security risks of traditional SMTP setups. Its robust features and seamless compatibility with Node.js and Express make it an ideal solution for supporting Village by CA's messaging needs.

Cloudinary

We chose Cloudinary for image management because it's a specialized, high-performance solution perfectly suited to the needs of a modern application. Cloudinary allows us to fully delegate image storage, optimization, and distribution, significantly reducing the load on our backend and improving overall website performance.

Images are automatically compressed, resized, and served via a CDN, ensuring fast loading and an excellent user experience. Cloudinary also offers a high level of security for uploads, as well as an API that's easy to integrate with Node.js, simplifying media management in our application.

Finally, the service is free for small projects and remains easily scalable, making it an ideal solution for an organization like Village by CA, which can evolve its platform without technical constraints.

JsonWebToken

We use the JSON Web Token (JWT) module for authentication because it's a modern, secure solution perfectly suited to REST APIs. JWT enables stateless authentication, meaning no server-side session storage, which simplifies the architecture and improves application scalability.

Each token contains essential user information (ID, role, expiration date) and is signed, guaranteeing its integrity and preventing fraudulent modification. This approach also simplifies role and permission management in our application. JWT is easy to integrate with Express, which is widely used in the industry and compatible with all platforms, making it a natural choice for securing access to our API.

Nodemailer

We use Nodemailer to manage email sending from our Express backend. It's a simple, reliable module perfectly suited to the Node.js ecosystem. Nodemailer makes it easy to send confirmation, contact, or notification emails without having to manually manage SMTP protocols, which greatly simplifies implementation.

The module is compatible with all email services (Gmail, Outlook, OVH, Mailgun, etc.), supports HTML emails, and offers secure configuration via TLS. Free, open-source, and widely used, Nodemailer is a robust and accessible solution, ideal for a non-profit project like Village by CA. We prefer it to express-mailer, which is more outdated and no longer maintained.

Validator

We use the validator module to ensure the quality and security of data sent to our API. Validator makes it easy to check sensitive formats (emails, passwords, field lengths, etc.) and prevent the recording of incorrect or potentially dangerous data.

It also offers sanitization features to clean user input (removing spaces, normalizing emails, escaping special characters), which improves the consistency of the data stored in the database. Lightweight, reliable, and widely used in the Node.js ecosystem, validator provides an essential security layer to protect the application and ensure clean and controlled processing of user information.

Bcrypt

We use Bcrypt to secure user passwords. Bcrypt is a hashing algorithm specifically designed for password protection. Passwords are therefore never stored in plain text in the database. Upon login, Bcrypt compares the provided password with the stored hash, ensuring secure authentication. Its robustness, ease of integration, and widespread adoption in the Node.js ecosystem make it a reliable and suitable solution for our project.

Multer and Multer-storage-cloudinary

We use Multer to manage file uploads in our Express API. Multer is a middleware specializing in processing forms containing files, which allows us to cleanly retrieve images sent by the frontend, control their size and type, and prevent upload errors.

To avoid storing files on our server, we also use multer-storage-cloudinary, which replaces local storage with automatic uploads to Cloudinary. As soon as a file is received, it is directly transferred to Cloudinary, which handles hosting, optimization, and distribution via the Content Delivery Network.

This combination results in a simple, secure, and efficient upload system, while keeping our backend lightweight and easy to maintain. Only the final URL of the image is stored in the database via Prisma, which significantly simplifies media management.

DEV dependencies :

Nodemon

We use Nodemon to improve the comfort and speed of developing our Express API. Nodemon automatically monitors project files and restarts the server as soon as a change is detected. This avoids manually restarting the server after each change and allows for a much smoother and more efficient development cycle.

The tool is easy to configure, widely used in the Node.js ecosystem, and perfectly suited to a modern development environment. Thanks to Nodemon, we save time and reduce the risk of errors associated with manual restarts. Furthermore, we have already used it and are therefore very familiar with this tool.

ESLint

We use ESLint to ensure the quality, consistency, and maintainability of the code in our project. ESLint automatically analyzes JavaScript/TypeScript code and detects potential errors, bad practices, or style inconsistencies. This helps prevent subtle bugs and ensures clean, consistent code, even when multiple developers are collaborating.

Design Justifications :

The design choices for the website were chosen to align with the company's established visual identity, starting with a color palette directly inspired by the tones and contrasts present in the existing logo. This continuity ensures a coherent brand experience across the company's various services.

A clean and minimalist design approach was intentionally chosen to offer users a modern and intuitive navigation environment, allowing key information to stand out effortlessly. Every decision was refined through multiple meetings with the client, ensuring that the final interface reflects their expectations.

Dependencies between tasks

1 : Create the database in order to be able to use the repo files that manipulate the data in the database.

→ Maï

2 : Create the repo files in order to use the services files so we know if the business logic is applied well.

→ Maï

3 : Create the configuration of Cloudinary so the services can upload into Cloudinary and save the picture data in the database so we can insert entities that use pictures.

→ Maï

4 : Create the services files so the controllers have functions to call, even if we can see if the controller is correctly working with just a simple response.

→ Guillaume and Maï

5 : Create the controllers files so the routes can return something via controllers.

→ Guillaume

6 : Create a login controller that generate JWT in order to use middlewares that verify the user role and the token presence.

→ Maï

7 : Finishing all the backend in order to display the data to the frontend.

→ Guillaume and Maï

Sprint Planning Overview

Here is the sprint plan used to create the Village by CA application, followed by detailed descriptions of each of them :

- Idea Developement
- Project Planning
- Technical Documentation
- Mockups Creation and Client Alignment
- Technical Knowledge Acquisition
- Database Setup and Basic Backend Foundation
- Repository Structure and Backend Completion
- Services Layer Implementation
- Frontend Technology Familiarization
- Frontend Progression and Bug Fixing

Sprint Table — Idea Development

Element	Details
Sprint	Idea Development
Sprint Duration	1 Week
Deadline	Sunday, Dec 7
Main Goal	Initial exploration of the concepts, identification of core objectives, and clarification of expected outcomes
Participants	Maï and Guillaume
Main Tasks	• Developers explore initial concepts and define the core product vision • Brainstorming several ideas: Maï and Guillaume • Ideas evaluation: Maï and Guillaume • Decision and refinement: Maï and Guillaume • SCM ensures early project structure and repository readiness • QA reviews feasibility aspects and identifies potential risks or constraints
Scrum Master	Maï Le Meur
Task Completion	3 out of 3
Sprint Velocity	100%

Sprint Retrospective — Idea Development

What worked well during the sprint

- Strong collaboration during brainstorming and idea evaluation
- Clear definition of the project vision and objectives
- Early SCM setup provided a solid repository foundation
- QA identified feasibility constraints early
- All tasks were completed successfully

What challenges did we face

- Initial ambiguity around the scope required extra clarification
- Aligning conceptual directions took additional discussion time

What changes can we make to improve the next sprint

- Document early decisions more systematically

Sprint Table — Project Planning

Element	Details
Sprint	Project Planning
Sprint Duration	1 Week
Deadline	Sunday, Dec 14
Main Goal	Definition of scope, task breakdown, prioritization, and sprint structuring to ensure a clear development roadmap.
Participants	Maï and Guillaume
Main Tasks	• Developers break down features into actionable tasks and estimate workload • Creation of a Gantt chart: Maï and Guillaume • SCM prepares branching strategies and repository conventions • QA defines the initial testing strategy and acceptance criteria
Scrum Master	Guillaume Leray Girardeau
Task Completion	2 out of 2
Sprint Velocity	100%

Sprint Retrospective — Project Planning

What worked well during the sprint

- Efficient breakdown of features into actionable tasks
- Smooth collaboration on the Gantt chart
- Full task completion with strong coordination

What challenges did we face

- Estimating workload for unfamiliar features required extra effort
- Aligning priorities across roles took time

What changes can we make to improve the next sprint

- Introduce estimation guidelines for consistency
- Hold a short alignment meeting before finalizing sprint scope

Sprint Table —Technical Documentation

Element	Details
Sprint	Technical Documentation
Sprint Duration	2 Weeks
Deadline	Sunday, Jan 11
Main Goal	Creation of foundational documentation, including architecture notes, workflow descriptions, and technical specifications.
Participants	Maï and Guillaume
Main Tasks	• Developers produce architectural diagrams, workflow descriptions, and technical specifications ◦ Define User Stories and Mockups: Guillaume and Maï ◦ Complete Figma Training: Guillaume ◦ Design System Architecture: Maï ◦ Define Components, Classes and Database Design: Maï ◦ Create High-level Sequence Diagram: Maï ◦ Document Internal and External API: Maï ◦ Plan SCM and QA Strategies: Guillaume • SCM validates documentation consistency with the planned development workflow • QA reviews documentation to ensure testability and clarity of requirements
Scrum Master	Maï Le Meur
Task Completion	6 out of 6
Sprint Velocity	100%

Sprint Retrospective —Technical Documentation

What worked well during the sprint

- Comprehensive documentation across architecture, workflows, APIs, and design
- Clear division of responsibilities between team members
- SCM and QA ensured consistency and testability
- All tasks were completed with high quality

What challenges did we face

- The large documentation volume required significant coordination
- Some technical decisions evolved mid-sprint and required updates

What changes can we make to improve the next sprint

- Adopt documentation versioning to track changes
- Schedule mid-sprint syncs to maintain alignment

Sprint Table — Mockup Creation & Client Alignment

Element	Details
Sprint	Mockup Creation & Client Alignment
Sprint Duration	1 Week
Deadline	Sunday, Jan 18
Main Goal	Design of interface mockups using Figma and validation with the client (Amandine Chemin) to ensure the proposed solution meets expectations.
Participants	Maï and Guillaume
Main Tasks	• Developers design UI mockups and refine them based on feedback ◦ Create visual mockups using Figma: Maï and Guillaume ◦ Hold a meeting with Amandine Chemin to establish a clear design direction: Maï and Guillaume • SCM stores and versions design assets in the repository • QA verifies that mockups meet usability expectations and match client requirements validated with Amandine Chemin
Scrum Master	Guillaume Leray Girardeau
Task Completion	2 out of 2
Sprint Velocity	100%

Sprint Retrospective — Mockup Creation & Client Alignment

What worked well during the sprint

- Effective creation of mockups using Figma
- Productive meeting with the client that clarified design expectations

What challenges did we face

- Some design elements required multiple iterations
- Limited time to explore alternative design options

What changes can we make to improve the next sprint

- Gather preliminary client input earlier
- Allocate buffer time for design iterations

Sprint Table — Technical Knowledge Acquisition

Element	Details
Sprint	Technical Knowledge Acquisition
Sprint Duration	2 Weeks
Deadline	Sunday, Feb 1
Main Goal	Learning and familiarization with required technologies such as Express, Node.js, Prisma, and Docker through official documentation.
Participants	Maï and Guillaume
Main Tasks	• Developers study Express, Node.js, Prisma, and Docker through official documentation ◦ Read documentation and complete a training course on Express and Node.js Guillaume ◦ Read documentation on Prisma: Maï ◦ Read documentation on Docker: Maï • SCM provides environment setup guidelines and ensures tools are correctly configured • QA prepares early test scenarios based on the technologies being adopted
Scrum Master	Maï Le Meur
Task Completion	3 out of 4
Sprint Velocity	75%

Sprint Retrospective — Technical Knowledge Acquisition

What worked well during the sprint

- Strong engagement with documentation and training materials
- Clear division of learning topics

What challenges did we face

- One task remained incomplete due to time constraints
- Some technologies required more learning time than expected

What changes can we make to improve the next sprint

- Allocate more time for deep technical learning earlier
- Use pair-learning sessions to accelerate understanding

Sprint Table — Database Setup & Backend Foundations

Element	Details
Sprint	Database Setup & Backend Foundations
Sprint Duration	1 Week
Deadline	Sunday, Feb 8
Main Goal	Creation of the database schema and implementation of initial backend components, including basic routes, controllers, and class structures.
Participants	Maï and Guillaume
Main Tasks	• Developers create the database schema and implement initial routes, controllers, and class structures ◦ Creation of basic API routes: Guillaume ◦ Implementation of basic controllers: Guillaume ◦ Initialization of the Database: Maï ◦ Development of the core class structure: Maï • SCM reviews early backend code and validates repository organization • QA tests basic endpoints using Postman and verifies database interactions
Scrum Master	Guillaume Leray Girardeau
Task Completion	3 out of 4
Sprint Velocity	75%
Bug Count	1 bug
Resolution Rate	100%

Sprint Retrospective — Database Setup & Backend Foundations

What worked well during the sprint

- Successful creation of the database schema and backend structure
- Good collaboration on routes, controllers, and classes
- SCM validated repository organization
- QA effectively tested endpoints and database interactions

What challenges did we face

- One task remained incomplete
- A bug was identified but resolved quickly

What changes can we make to improve the next sprint

- Improve time estimation for backend setup tasks
- Introduce early code reviews to catch issues sooner

Sprint Table — Repository Structure & Backend Completion

Element	Details
Sprint	Repository Structure & Backend Completion
Sprint Duration	1 Week
Deadline	Sunday, Feb 15
Main Goal	Setup of repository files and completion of backend classes, controllers, and structural elements.
Participants	Maï and Guillaume
Main Tasks	• Developers set up the repository files and complete the initial backend components, including classes, controllers, and core structural elements ◦ Final classes creation: Maï ◦ Finishing controllers implementation: Guillaume • SCM reviews the repository organization and validates backend code structure before merging into the development branch • QA performs early functional checks to ensure that backend routes, controllers, and structural elements behave as expected
Scrum Master	Maï Le Meur
Task Completion	3 out of 5
Sprint Velocity	60%
Bug Count	8 bugs
Resolution Rate	100%

Sprint Retrospective — Repository Structure & Backend Completion

What worked well during the sprint

- Repository structure was properly set up and validated
- Backend classes and controllers progressed significantly
- QA performed early functional checks
- All identified bugs were resolved

What challenges did we face

- Only 60% of tasks were completed due to complexity
- High bug count indicated structural issues

What changes can we make to improve the next sprint

- Focus on backend consistency before expanding features
- Implement stricter coding standards to reduce bugs

Sprint Table — Service Layer Implementation

Element	Details
Sprint	Service Layer Implementation
Sprint Duration	1 Week
Deadline	Sunday, Feb 22
Main Goal	Development of service modules and integration with existing controller logic to support business operations.
Participants	Maï and Guillaume
Main Tasks	• Developers implement service modules and connect them to controllers ◦ Services file creation: Maï and Guillaume ◦ Implementation of prisma transaction: Guillaume ◦ Connecting services to controllers: Maï • SCM reviews service logic and validates integration with existing backend components • QA tests service endpoints and verifies business logic correctness
Scrum Master	Guillaume Leray Girardeau
Task Completion	4 out of 5
Sprint Velocity	80%
Bug Count	33 bugs
Resolution Rate	85%

Sprint Retrospective — Service Layer Implementation

What worked well during the sprint

- Service modules were successfully created and integrated
- Prisma transactions were implemented effectively
- SCM validated service logic
- QA tested business logic thoroughly

What challenges did we face

- One task remained incomplete
- High bug count (33) indicated complexity in service logic
- Some integration points required rework

What changes can we make to improve the next sprint

- Improve service design before implementation
- Increase pair programming to reduce inconsistencies

Sprint Table — Frontend Technology Familiarization

Element	Details
Sprint	Frontend Technology Familiarization
Sprint Duration	1 Week
Deadline	Sunday, Mar 1
Main Goal	Introduction to the frontend stack, implementation of Cloudinary and JWT, creation of authentication middlewares, and development of the contact form.
Participants	Maï and Guillaume
Main Tasks	• Developers explore the frontend stack with React and Vite and implement Cloudinary, JWT, authentication middlewares, and the contact form ◦ Read documentation and complete a training course on React and Vite: Guillaume ◦ Create basic frontend routes and architecture: Guillaume ◦ Read documentation about Cloudinary: Maï ◦ Implement authentication with JWT: Maï ◦ Add contact form to the frontend: Maï • SCM ensures proper frontend folder structure and asset management • QA tests authentication flows, media uploads, and form submissions
Scrum Master	Maï Le Meur
Task Completion	3 out of 4
Sprint Velocity	75%
Bug Count	21 bugs
Resolution Rate	95%

Sprint Retrospective — Frontend Technology Familiarization

What worked well during the sprint

- Strong progress in learning React, Vite, Cloudinary, and JWT
- Successful implementation of authentication and the contact form
- SCM ensured a clean frontend structure
- QA validated authentication flows and media uploads

What challenges did we face

- One task remained incomplete
- Integration of new technologies required extra debugging

What changes can we make to improve the next sprint

- Provide more time for frontend onboarding
- Create small prototype components before full integration

Sprint Table — Frontend Progression & Bug Fixing

Element	Details
Sprint	Frontend Progression & Bug Fixing
Sprint Duration	1 Week
Deadline	Sunday, Mar 8
Main Goal	Advancement of frontend features, implementation of formatting utilities, and correction of identified bugs.
Participants	Maï and Guillaume
Main Tasks	• Developers advance UI implementation, add formatting functions, and fix identified bugs. ◦ Connect contact form to the backend: Maï ◦ Creating React components: Guillaume ◦ Adding missing frontend pages with new components: Maï ◦ Creation of a test suite with Jest: Maï • SCM reviews frontend code and ensures consistency with coding standards. • QA validates UI behavior, checks formatting logic, and confirms bug resolutions.
Scrum Master	Guillaume Leray Girardeau
Task Completion	3 out of 4
Sprint Velocity	75%
Bug Count	7 bugs
Resolution Rate	85%

Sprint Retrospective — Frontend Technology Familiarization

What worked well during the sprint

- Significant progress on UI components and missing pages
- Contact form successfully connected to the backend
- Jest test suite creation improved testing capabilities
- QA validated UI behavior and bug fixes

What challenges did we face

- One task remained incomplete
- Some bugs required multiple iterations

What changes can we make to improve the next sprint

- Increase automated testing coverage to reduce regressions

Project Closure Report

Project Retrospective – Village by CA

1. Results & Summary

Key MVP Features

The delivered MVP covers the essential needs defined in the User Stories:

- **Showcase website for the association**
 - Presentation of the association and its members
 - Dedicated pages for presenting startups, partners, facilities, and news from the Village by CA
 - Highlighting startups, quotes, and news on the homepage
- **Contact form**
 - Sending the contact form by email to the administrator
 - Implementation of a predefined list of email subjects to facilitate topic identification and improve the readability of incoming requests.
- **Authentication**
 - Secure login with JWT issuance
 - Role management (admin / user)
- **Complete back-office for the administrator**
 - A dedicated management space for each entity: creation, update, deletion.

Comparison with initial objectives

Feature	Result	Comment
Homepage with presentation of the association, members, startups, display of quotes, news and events	85%	Everything is there, it's just missing a calendar display for events.
A page dedicated to showcasing startups, displaying all their specific information with filters for dynamic display and an easier navigation.	100%	Everything is there and it is working.
A page dedicated to showcasing partners, displaying all their specific informations, with a separation between partners who give a financial aid and those who don't.	100%	Everything is there and it is working.
A page dedicated to showcasing the premises and displaying the association's rooms for rent.	100%	Everything is there and it is working.
A page dedicated to association news, highlighting upcoming events and displaying all articles about the life of the association.	80%	Everything is there, it's just missing a calendar display for events.
A contact form so that the site visitor has the opportunity to ask their questions or get in touch with the association with predefined discussion topics so that the recipient (the admin) has better visibility on the subject of the email.	100%	Everything is there and it is working.
A login area that redirects to the administrator area if it is an admin or to the profile area of the logged-in user.	100%	Everything is there and it is working.

Feature	Result	Comment
A profile section of the current user, listing all user information: name and email address. Their startup and the ability to edit all their information. Their quotes and the ability to edit, create, and delete them.	100%	Everything is there and it is working.
An admin area where all entities that can be manipulated by an administrator are listed, which redirects to the area dedicated to that purpose.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more users.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more startups.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more partners.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more posts.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more types.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more locations.	100%	Everything is there and it is working.
A space that allows an admin to create, modify, view and delete one or more quotes.	100%	Everything is there and it is working.

Even though the project isn't in production, we can identify some internal metrics:

- 95% of MVP features delivered
 - Significant backend test coverage (critical services secured)
 - Zero blocking bugs at the end of the sprint
 - Complete documentation covering 100% of modules
 - UI created with the client
-
- Deadlines met despite unforeseen technical challenges and personal challenges

2. Lessons learned

What worked well

- Smooth and structured collaboration
- Regular exchanges, documented decisions and transparency have allowed us to move forward quickly and with clear objectives.
- Creating a solid architecture
- Working on the application architecture allowed us to gain a clearer understanding of the technologies we needed to master and to know the specific purpose of each technology, in order to address all the challenges we faced. This enabled us to move forward more quickly, as all the technologies were defined beforehand.
- Creation of mockups for the website's UI
- This allowed us to move faster in creating the Frontend during the development phase because we had created the visuals upstream and in collaboration with the client.
- Documentation
- It served as a common reference point and facilitated communication with the client, ensuring that objectives were aligned with the client's needs.
- Effective bug fixing
- Methodical approach, progressive investigation, targeted tests and manual tests.

Challenges encountered and solutions implemented

- Initial ambiguities regarding certain features
- → Resolved through regular exchanges with the client, reformulations, and documentation of decisions.
- Receiving image files to send to Cloudinary
- → Resolved by a change of strategy, switching from sending base64 images to using a multer middleware that receives binary data from files.
- Time management on new technical topics
- → Resolved through rapid skills development and strict prioritization.
- Obtain OAuth2 credentials so that an email address can be used to submit a form.
- → Resolved through discussions with peers and Gmail documentation.

Areas for improvement for future projects

- Better time management
- Perhaps reduce the time for learning new technologies or allocate more leeway for the end of the project in case of unforeseen events to be less rushed/stressed at the end of the project and have more time to refine it.
- The structure of our frontend and our CSS files.
- With React, we create a component file and an associated CSS file. However, some components reuse the same CSS, leading to code repetition. We need to be more organized upstream and identify the components that reuse the same properties.
- Add more time for testing.
- Even though the project doesn't have any major/blocking bugs and has undergone a lot of unit and manual testing, an extra day or two to find more hidden/minor bugs would have been ideal.
- Continue to structure sprints with measurable objectives
- This worked very well, and can be further improved.

3- Conclusion

The Village by CA project has a 90% complete MVP, a clean architecture, a UI aligned with the client's vision, and comprehensive documentation.

The challenges were overcome through effective collaboration, technical rigor, and adaptability.

This project allowed us to learn new technologies and develop our technical and project management skills.